-

:

«- »

:

« Pixhawk»

: . .

: . .

: 3331506/30401

-

2026

Contents

— [2]. (IMU — Inertial Measurement Unit), . , : , [1].

Pixhawk [10]. -IMU MAVLink [8] — , [9].

, IMU- (RAW_IMU MAVLink) Pixhawk : [3], [6]. 3D- OpenGL 3.3 GLFW [11].

:

1. MAVLink ;

2. ;

3. ;

4. ZUPT (Zero Velocity Update) ;

5. OpenGL- 3D-, HUD- ;

6. .

1 Pixhawk

1.1 MAVLink RAW_IMU

MAVLink [8]. MAVLink (Micro Air Vehicle Link) — , 2009 [9]. PX4 ArduPilot [10]. RAW_IMU (27 common) [9]. (LSB — Least Significant Bit):

Table 1: MAVLink RAW_IMU

xacc, yacc, zacc	X, Y, Z (LSB)
xgyro, ygyro, zgyro	(LSB)
xmag, ymag, zmag	(LSB)

1.2

(pixhawk_imu_reader_simple.cpp), Pixhawk . /dev/ttyACM0 115 200 . POSIX- termios [12]: 8 , , -, , 1 .

```
int setup_serial_port(const char* port, int baudrate) {
    int fd = open(port, O_RDWR | O_NOCTTY);
    struct termios tty;
    memset(&tty, 0, sizeof(tty));
    tcgetattr(fd, &tty);
    cfsetospeed(&tty, baudrate);
    cfsetispeed(&tty, baudrate);
    tty.c_cflag |= (CLOCAL | CREAD);
    tty.c_cflag &= ~CSIZE;
    tty.c_cflag |= CS8;          // 8 data bits
    tty.c_cflag &= ~PARENB;      // no parity
    tty.c_cflag &= ~CSTOPB;      // 1 stop bit
    tty.c_cflag &= ~CRTSCTS;     // no hardware flow control
    tty.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
    tty.c_iflag &= ~(IXON | IXOFF | IXANY);
    tty.c_oflag &= ~OPOST;
    tty.c_cc[VMIN]  = 0;
    tty.c_cc[VTIME] = 10;        // 1 second timeout
    tcsetattr(fd, TCSANOW, &tty);
```

```c
    return fd;
}
```

Listing 1: termios

MAVLink- : mavlink_parse_char() , CRC-16 [8].

```c
mavlink_message_t msg;
mavlink_status_t  status;
uint8_t buffer[2048];

while (running) {
    int n = read(serial_fd, buffer, sizeof(buffer));
    if (n > 0) {
        for (int i = 0; i < n; i++) {
            if (mavlink_parse_char(MAVLINK_COMM_0,
                                    buffer[i], &msg, &status)) {
                if (msg.msgid == MAVLINK_MSG_ID_RAW_IMU) {
                    mavlink_raw_imu_t imu;
                    mavlink_msg_raw_imu_decode(&msg, &imu);
                    handle_message(&imu);
                }
            }
        }
    }
}
```

Listing 2: MAVLink-

1.3 CSV

CSV- , gettimeofday() . (flush()) 50 — -. : timestamp, AX_raw, AY_raw, AZ_raw, GX_raw, GY_raw, GZ_raw, MX_raw, MY_raw, MZ_raw.

CSV (bias) .

1.4

RAW_IMU 100 , IMU Pixhawk [10]. Z ( ), — , .

2

2.1

, , Pixhawk IMU Visualizer — C++14, .

() : Pixhawk, RAW_IMU , CSV-.

() : , .

2.2

Table 2:

imu_shared.h	: RawIMU, NavState, SharedBuffer,
imu_visualizer.h	run_visualizer()
imu_visualizer.cpp	, GLSL-, OpenGL-
pixhawk_imu_reader.cpp	MAVLink-,
CMakeLists.txt	CMake
c_library_v2/	MAVLink C [9]

2.3

, imu_shared.h.

RawIMU :

```
struct RawIMU {
    double  timestamp;
    int16_t xacc, yacc, zacc;      //   (LSB)
    int16_t xgyro, ygyro, zgyro;   //   (LSB)
    int16_t xmag, ymag, zmag;      //   (LSB)
};
```

Listing 3: IMU

NavState — , , , , :

```
struct NavState {
    double qw=1, qx=0, qy=0, qz=0; //
    double wx=0, wy=0, wz=0;        //   (/)
    double vx=0, vy=0, vz=0;        //   (/)
```

```
    double px=0, py=0, pz=0;          //  ()
    double roll=0, pitch=0, yaw=0;    //   ()
    double ax=0, ay=0, az=0;          //  (/^2)
};
```

Listing 4:

SharedBuffer :

```
struct SharedBuffer {
    std::mutex              mtx;
    std::deque<RawIMU>      queue;          //    (.  500)
    static const int        MAX_QUEUE = 500;
    std::atomic<bool>       running{true};
    std::mutex              nav_mtx;
    NavState                nav;            //
    std::mutex              trail_mtx;
    std::deque<std::array<float,3>> trail; //   (.  2000  )
    static const int        MAX_TRAIL = 2000;

    void push(const RawIMU& r){ /*    */ }
    bool pop(RawIMU& out)      { /*    */ }
    void add_trail_point(float x, float y, float z) { /* ... */ }
};
```

Listing 5:

2.4

SharedBuffer , . running std::atomic<bool>, — SIGINT (Ctrl+C).

3

3.1

Table 3:

MAVLink c_library_v2 [9]	2.0	, RAW_IMU
GLFW3 [11]	≥3.3	OpenGL,
GLEW	≥2.1	OpenGL 3.3
GLM	≥0.9.9	
pthreads	POSIX	(std::thread)

MAVLink — .h- [8]. Ubuntu.

3.2 CMake

CMake 3.10. CMakeLists.txt C++14, find_package PkgConfig -O2 -Wall -Wextra.

4

4.1

- (bias), [1]. . :

$$a_i = \left(\tilde{a}_i - b_{a,i}\right) \cdot S_a, \quad \omega_i = \left(\tilde{\omega}_i - b_{\omega,i}\right) \cdot S_\omega, \tag{1}$$

$\tilde{a}_i$, $\tilde{\omega}_i$ — ; $b_{a,i}$, $b_{\omega,i}$ — ; $S_a = 9{,}81/3050$ $[/^2/]$, $S_\omega = 0{,}001$ $[//]$. , (CSV-):

$$\mathbf{b}_a = (31,\ 24,\ 0)^\top, \quad \mathbf{b}_\omega = (-7,\ 1{,}5,\ 3)^\top. \tag{2}$$

Calib imu_shared.h, .

4.2

[3]:

$$\mathbf{q} = q_w + q_x\,\mathbf{i} + q_y\,\mathbf{j} + q_z\,\mathbf{k}, \quad \|\mathbf{q}\| = 1. \tag{3}$$

: (gimbal lock), (4 9) [3].

4.3

$\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)^\top$ [4]:

$$\dot{\mathbf{q}} = \frac{1}{2}\,\mathbf{q} \otimes \begin{pmatrix} 0 \\ \boldsymbol{\omega} \end{pmatrix}. \tag{4}$$

Δt -, [1]:

$$\theta = \|\boldsymbol{\omega}\| \cdot \Delta t, \qquad \Delta\mathbf{q} = \left(\cos\frac{\theta}{2},\ \frac{\boldsymbol{\omega}}{\|\boldsymbol{\omega}\|}\sin\frac{\theta}{2}\right). \tag{5}$$

:

$$\mathbf{q} \leftarrow \frac{\mathbf{q} \otimes \Delta\mathbf{q}}{\|\mathbf{q} \otimes \Delta\mathbf{q}\|}. \tag{6}$$

, $\mathbf{q}$ [4].

4.4

$\mathbf{R} \in SO(3)$ [3]:

$$\mathbf{R} = \begin{pmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{pmatrix}. \tag{7}$$

4.5 (ZYX)

ϕ, θ ψ [1]:

$$\phi = \operatorname{atan2}\bigl(2(q_w q_x + q_y q_z),\ 1 - 2(q_x^2 + q_y^2)\bigr), \tag{8}$$

$$\theta = \arcsin\bigl(2(q_w q_y - q_z q_x)\bigr), \tag{9}$$

$$\psi = \operatorname{atan2}\bigl(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\bigr). \tag{10}$$

4.6 ( )

[6]:

$$\mathbf{a}_{\text{world}} = \mathbf{R}\,\mathbf{a}_{\text{body}} - \mathbf{g}_{\text{world}}, \quad \mathbf{g}_{\text{world}} = (0,\ 0,\ -9{,}81)^\top\ [/^2]. \tag{11}$$

[2]:

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a}_{\text{world}}(t) \cdot \Delta t, \tag{12}$$

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t) \cdot \Delta t. \tag{13}$$

4.7 ZUPT

[2]. ZUPT (Zero Velocity Update) — INS [7]. , :

$$\|\boldsymbol{\omega}\| < \omega_{\text{th}} : \quad \mathbf{v} \leftarrow \mathbf{v} \cdot \alpha_{\text{damp}}, \tag{14}$$

$\omega_{\text{th}} = 0{,}015$ / — (Calib::ZUPT_GYRO_THRESH), $\alpha_{\text{damp}} = 0{,}85$ — . ZUPT 0,3 $/^2$, .

5

5.1 IMU

integrate_imu() imu_visualizer.cpp. .

(1):

```
double ax = (raw.xacc  - Calib::AX_BIAS) * Calib::ACCEL_SCALE;
double ay = (raw.yacc  - Calib::AY_BIAS) * Calib::ACCEL_SCALE;
double az = (raw.zacc  - Calib::AZ_BIAS) * Calib::ACCEL_SCALE;
double gx = (raw.xgyro - Calib::GX_BIAS) * Calib::GYRO_SCALE;
double gy = (raw.ygyro - Calib::GY_BIAS) * Calib::GYRO_SCALE;
double gz = (raw.zgyro - Calib::GZ_BIAS) * Calib::GYRO_SCALE;
```

Listing 6:

(5)–(6):

```
double theta = std::sqrt(gx*gx + gy*gy + gz*gz) * dt;
if (theta > 1e-10) {
    double norm_w = std::sqrt(gx*gx + gy*gy + gz*gz);
    double s   = std::sin(theta * 0.5) / norm_w;
    double c   = std::cos(theta * 0.5);
    double dqw = c, dqx = gx*s, dqy = gy*s, dqz = gz*s;

    // q_new = q (x) dq
    double qw = n.qw*dqw - n.qx*dqx - n.qy*dqy - n.qz*dqz;
    double qx = n.qw*dqx + n.qx*dqw + n.qy*dqz - n.qz*dqy;
    double qy = n.qw*dqy - n.qx*dqz + n.qy*dqw + n.qz*dqx;
    double qz = n.qw*dqz + n.qx*dqy - n.qy*dqx + n.qz*dqw;

    double norm = std::sqrt(qw*qw + qx*qx + qy*qy + qz*qz);
    n.qw = qw/norm; n.qx = qx/norm;
    n.qy = qy/norm; n.qz = qz/norm;
}
```

Listing 7: (-)

(7), , ZUPT (14):

```
//
double r00 = 1-2*(qy*qy+qz*qz), r01 = 2*(qx*qy-qw*qz), ...;
//
double ax_w = r00*ax + r01*ay + r02*az;
double ay_w = r10*ax + r11*ay + r12*az;
```

```
double az_w = r20*ax + r21*ay + r22*az;
az_w -= (-9.81);    //

// ZUPT
double gyro_norm = std::sqrt(gx*gx + gy*gy + gz*gz);
if (gyro_norm < Calib::ZUPT_GYRO_THRESH) {
    n.vx *= 0.85; n.vy *= 0.85; n.vz *= 0.85;
    if (std::abs(ax_w) < 0.3) ax_w = 0;
    if (std::abs(ay_w) < 0.3) ay_w = 0;
    if (std::abs(az_w) < 0.3) az_w = 0;
}
//  v  p
n.vx += ax_w * dt;  n.vy += ay_w * dt;  n.vz += az_w * dt;
n.px += n.vx * dt;  n.py += n.vy * dt;  n.pz += n.vz * dt;
```

Listing 8: ZUPT

5.2

(8)–(10) :

```
const double R2D = 180.0 / M_PI;
n.roll  = std::atan2(2*(qw*qx + qy*qz),
                     1-2*(qx*qx + qy*qy)) * R2D;
double sp = std::max(-1.0, std::min(1.0, 2*(qw*qy - qz*qx)));
n.pitch = std::asin(sp) * R2D;
n.yaw   = std::atan2(2*(qw*qz + qx*qy),
                     1-2*(qy*qy + qz*qz)) * R2D;
```

Listing 9: (ZYX)

asin $[-1, 1]$ - .

6

6.1 OpenGL 3.3

GLSL-. (VERT3D / FRAG3D) 3D- :

```
#version 330 core
layout(location=0) in vec3 aPos;
layout(location=1) in vec3 aColor;
layout(location=2) in vec3 aNormal;
out vec3 vColor; out vec3 vNormal; out vec3 vPos;
uniform mat4 uMVP; uniform mat4 uModel;
void main(){
    vPos    = vec3(uModel * vec4(aPos,1));
    vNormal = mat3(transpose(inverse(uModel))) * aNormal;
    vColor  = aColor;
    gl_Position = uMVP * vec4(aPos,1);
}
```

Listing 10: 3D-

```
#version 330 core
in vec3 vColor; in vec3 vNormal; in vec3 vPos;
out vec4 outColor;
uniform vec3 uLight; uniform float uAlpha;
void main(){
    float d = max(dot(normalize(vNormal), normalize(uLight)), 0.0)
        ;
    outColor = vec4(vColor * (0.35 + 0.65*d), uAlpha);
}
```

Listing 11: 3D-

(VERT_FLAT / FRAG_FLAT) : , , HUD-. — .

6.2

3D- FC . push_box() :

```
static std::vector<V3> build_fc()
{
    std::vector<V3> v;
    //    -
    push_box(v,  0,0,0,     0.8f,0.8f,0.14f,  0.13f,0.14f,0.19f);
```

```
    //  кнь ( Y+)
    push_box(v, 0,0.37f,0.08f, 0.7f,0.06f,0.04f, 0.95f,0.95f,0.95f);
    //
    push_box(v, 0.37f,0,0.08f, 0.04f,0.8f,0.04f, 0.22f,0.22f,0.28f);
    push_box(v,-0.37f,0,0.08f, 0.04f,0.8f,0.04f, 0.22f,0.22f,0.28f);
    // : X=, Y=, Z=
    push_arrow(v,0,0,0.1f, 1,0,0, 0.7f, 0.95f,0.10f,0.10f);
    push_arrow(v,0,0,0.1f, 0,1,0, 0.7f, 0.10f,0.92f,0.10f);
    push_arrow(v,0,0,0.07f,0,0,1, 0.7f, 0.15f,0.42f,0.98f);
    return v;
}
```

Listing 12: FC

6.3 HUD-

RingBuf — std::deque. $N = 300$ push() :

```
struct RingBuf {
    std::deque<float> d;
    int maxN;
    explicit RingBuf(int n=300): maxN(n){}
    void push(float v){
        d.push_back(v);
        if((int)d.size()>maxN) d.pop_front();
    }
    float vmin() const { float m=1e9f;  for(auto x:d) m=std::min(m,x); return m; }
    float vmax() const { float m=-1e9f; for(auto x:d) m=std::max(m,x); return m; }
};
```

Listing 13:

12 : , , .

6.4

run_visualizer() GLFW/GLEW, , . :

1. (,).
2. SharedBuffer , integrate_imu(), .
3. . : FC, () .
4. 3D- (72%): , , , FC.
5. 2D HUD (30%): , , .

FC GLM:

```
glm::mat4 fc_pos = glm::translate(glm::mat4(1),
     glm::vec3((float)nav.px,(float)nav.py,(float)nav.pz));
glm::quat q((float)nav.qw,(float)nav.qx,
            (float)nav.qy,(float)nav.qz);
glm::mat4 fc_model = fc_pos * glm::mat4_cast(q);
glm::mat4 fc_mvp   = vp * fc_model;
```

Listing 14:

: , . :

```
float fade_step = 1.0f / buf.trail.size();
for(int i = 0; i < (int)buf.trail.size()-1; i++){
    float alpha = (float)i * fade_step;
    float cr = 0.1f + 0.7f * alpha;  // : 0.1 -> 0.8
    float cg = 0.5f + 0.4f * alpha;  // : 0.5 -> 0.9
    float cb = 0.9f;                 // :
    //   trail[i]..trail[i+1]   (cr,cg,cb)
}
```

Listing 15:

6.5

glViewport():

Table 4:

(3D-)	72%	FC, , ,
(HUD)	30%	4

— 1400×860 . GLFW g_W, g_H glViewport().

7

7.1

Ubuntu:

```
sudo apt update
sudo apt install build-essential cmake git \
    libglfw3-dev libglew-dev libglm-dev libgl1-mesa-dev
```

MAVLink — , :

```
cd pixhawk_imu_visualizer
git clone https://github.com/mavlink/c_library_v2.git
```

7.2

```
mkdir build && cd build
cmake ..
make -j$(nproc)
```

build/ pixhawk_imu_reader.

7.3

sudo dialout:

```
sudo usermod -a -G dialout $USER
# , :
sudo chmod 666 /dev/ttyACM0
```

7.4

Pixhawk USB /dev/ttyACM0 [10].

```
cd build
./pixhawk_imu_reader
```

. Ctrl+C .

7.5

Table 5:

+	
/	(0.5 — 30)
R	$(0, 0, 0)$
Ctrl+C	

8

8.1

. . , (8)–(10), . - [4].
HUD- . , (2).

8.2

. — : 5–10 , . [2].
ZUPT , - (10^{-3}–10^{-2} $/^2$).

8.3

— [1, 2]. $a(t)$ $\hat{a}(t)$:

$$\hat{a}(t) = a(t) + b + \eta(t), \tag{15}$$

b — , $\eta(t)$ — . :

$$\delta p(T) = \frac{1}{2}\, b\, T^2 + \int_0^T (T - t)\, \eta(t)\, dt. \tag{16}$$

[6]. - Pixhawk 10^{-3}–10^{-2} $/^2$, 10 [2].

8.4

[6]. :

1. GPS- [6] — INS. GPS 1–10 ; IMU — 100–1000 .
2. [5] — , . , .
3. SO(3) [4] — .
4. [10] — ; GPS ().
5. ZUPT [7] — .

Pixhawk. :

- RAW_IMU MAVLink [9] CSV ;
- ;
- - [3] ;
- ;
- [6];
- ZUPT [7] ;
- C++14 ;
- OpenGL 3.3- GLSL-, 3D- , HUD- [11].

: . , - [1, 2]: 5–10 . [6]: IMU . .

References

[1] Titterton D., Weston J. Strapdown Inertial Navigation Technology. 2nd ed. — IET, 2004. — 558 p.

[2] Woodman O. J. An Introduction to Inertial Navigation. Technical Report UCAM-CL-TR-696. — University of Cambridge, 2007. — 37 p. — URL: https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf (. 01.03.2024).

[3] Kuipers J. B. Quaternions and Rotation Sequences. — Princeton University Press, 1999. — 398 p.

[4] Mahony R., Hamel T., Pflimlin J.-M. Nonlinear Complementary Filters on the Special Orthogonal Group // IEEE Transactions on Automatic Control. — 2008. — Vol. 53, No. 5. — P. 1203–1218. — DOI: 10.1109/TAC.2008.923738.

[5] Madgwick S. O. H., Harrison A. J. L., Vaidyanathan R. Estimation of IMU and MARG orientation using a gradient descent algorithm // IEEE International Conference on Rehabilitation Robotics (ICORR). — 2011. — P. 1–7. — DOI: 10.1109/ICORR.2011.5975346.

[6] Groves P. D. Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems. 2nd ed. — Artech House, 2013. — 776 p.

[7] Skog I., Handel P. In-Car Positioning and Navigation Technologies — A Survey // IEEE Transactions on Intelligent Transportation Systems. — 2009. — Vol. 10, No. 1. — P. 4–21. — DOI: 10.1109/TITS.2008.2011712.

[8] Meier L. et al. MAVLink: A Lightweight Message Marshalling Library for the DroneCode Ecosystem. — URL: https://github.com/mavlink/mavlink (. 01.03.2024).

[9] MAVLink Developer Guide: Common Messages. — URL: https://mavlink.io/en/messages/common.html (. 01.03.2024).

[10] PX4 Autopilot User Guide. — URL: https://docs.px4.io/main/en/ (. 05.03.2024).

[11] GLFW — An OpenGL library. Documentation. — URL: https://www.glfw.org/docs/latest/ (. 05.03.2024).

[12] The Open Group. The Single UNIX Specification, Version 4. termios.h — define values for termios. — URL: https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/termios.h.html (. 01.03.2024).